

Monte Carlo AIXI with Context Tree Weighting

Chapter 12

Slides based on: Marcus Hutter, Elliot Catt, David Quarel

An Introduction to Universal Artificial Intelligence, CRC Press, 2024, Chapter 12

"The best way to predict your future is to create it." — Peter Drucker

Outline

- 1 Learning and Searching
- 2 Searching via Monte Carlo Tree Search
- 3 Learning via Context Tree Weighting
- 4 All Together
- 5 Experiments
- 6 AIXIjs Implementation
- 7 Discussion
- 8 Exercises
- 9 History and References

From AIXI-MDP to a More General Approximation I

- Chapter 11 introduced **AIXI-MDP**, an approximation of AIXI that performs a fixed-depth expectimax search but only over the class of all Markov environments as its model class. For small $\mathcal{O} = \mathcal{A} = \mathbb{B}$, each environment was encoded as a 4-tuple $(\theta_{00}, \theta_{01}, \theta_{10}, \theta_{11})$ learned by a KT estimator.
- This chapter builds a strictly more powerful approximation of AIXI, using as the model class $\mathcal{M} = \mathcal{C}_D$, the set of *all variable-order Markov environments with context length at most D* (recall D is a fixed maximum context depth).
- We call the resulting agent **Monte Carlo AIXI with Context Tree Weighting (MC-AIXI-CTW)**. It replaces the two computationally hard parts of AIXI with tractable approximations:
 - **Searching**: the exact expectimax operation is replaced by a Monte Carlo Tree Search (MCTS) variant.
 - **Learning**: the incomputable Bayes mixture ξ over *all* semicomputable environments is replaced by a Bayesian mixture over context trees, computed efficiently using **Context Tree Weighting (CTW)** (Chapter 4).

From AIXI-MDP to a More General Approximation II

- Moving from a limited-depth expectimax search to MCTS lets the agent plan to much greater depths, and MCTS is asymptotically as good as full expectimax search. Using CTW for prediction lets the agent model *any* k -Markov environment for any $k \leq D$, where D is the maximum depth of the CTW tree.

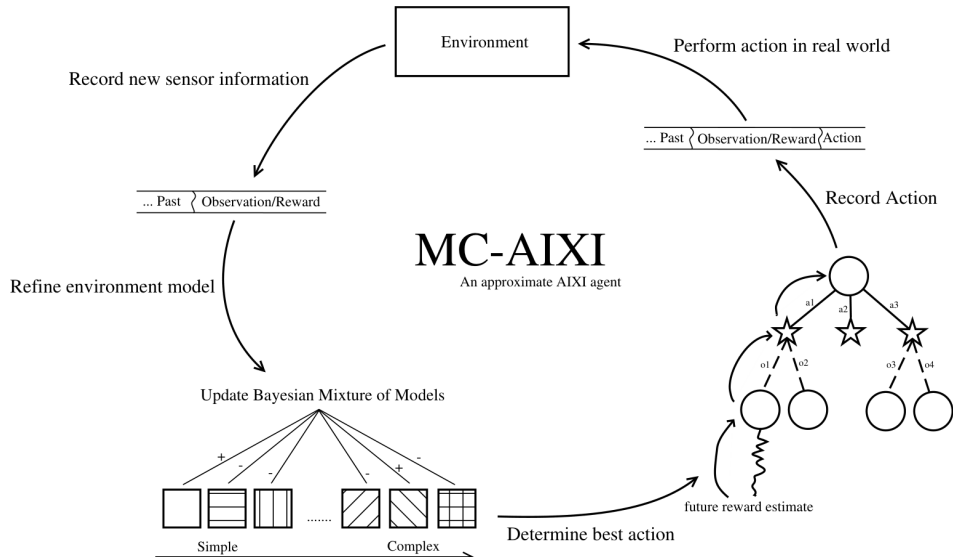
Definition: Action-Observation Search Tree

Definition 12.1.1 (Action-observation search tree)

An *action-observation search tree* is a tree comprised of alternating *action nodes* and *chance nodes*. Each node is associated with a history h . Additionally, action (resp. chance) nodes store an estimate of the value $\hat{V}(h)$ (resp. Q-value $\hat{Q}(h, a)$) of the associated history h (resp. ha).

Plain-English summary. Think of the tree of everything that *could* happen from now on: at an action node the agent chooses an action a (white node in Figure 12.2 below); at the resulting chance node the environment “rolls the dice” and returns an observation/reward pair (black node). $\hat{V}(h)$ is the agent’s current best guess at how good the situation described by history h is, and $\hat{Q}(h, a)$ is its best guess at how good it is to take action a having reached history h . Figure 12.1 (next slide) shows how this tree fits into the whole agent–environment loop of MC-AIXI-CTW.

Figure 12.1: The MC-AIXI-CTW Interaction Loop



Recap: The AIXI Expectimax Operation

Recall that the action AIXI takes maximizes the (undiscounted) expected value in the mixture environment ξ , given by the expectimax Theorem 7.4.2:

AIXI Policy (Theorem 7.4.2)

$$\pi^{\text{AIXI}}(h_{<t}) := \arg \max_{a_t} \sum_{e_t} \cdots \max_{a_{t+m}} \sum_{e_{t+m}} \left(\sum_{i=t}^{t+m} r_i \right) \sum_{\nu \in \mathcal{M}} w(\nu|h_{<t}) \prod_{k=t}^{t+m} \nu(e_k|h_{<k}a_k)$$

Plain-English summary. π^{AIXI} is the policy (rule for choosing actions) that AIXI follows. $h_{<t}$ is the history of actions/observations/rewards seen before time t . The agent uses a *forward moving horizon*: it always looks exactly m steps into the future from the current time step. e_k is a percept (observation+reward pair) at time k ; the sums over $e_t, \dots$ range over all possible future percept sequences, while the $\max_{a_k}$ range over all possible future actions. ν (nu) ranges over environments $\nu \in \mathcal{M}$ in the model class, $w(\nu|h_{<t})$ is the posterior weight (belief) the agent assigns to environment ν given the history so far, and $\prod_k \nu(e_k|h_{<k}a_k)$ is the probability that ν would generate the imagined future percept sequence. In words: try every action, imagine every possible consequence, weight each scenario by how much the agent currently believes in the environment producing it, and pick the action that maximizes expected total reward $\sum r_i$.

Figure 12.2: Visualizing the Expectimax Tree

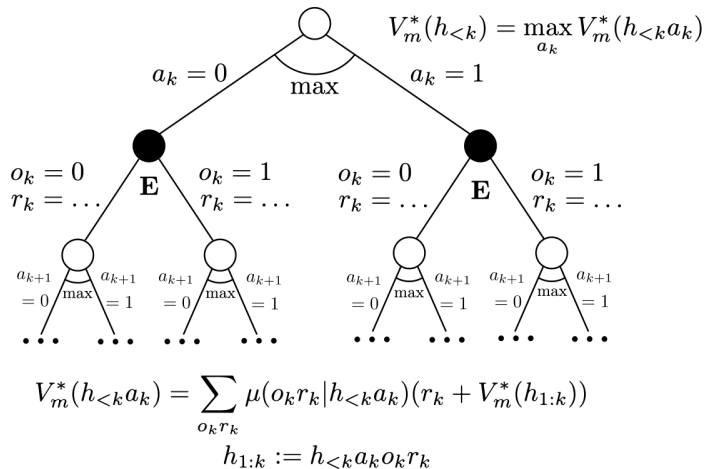


Figure 12.2: A visualization of the expectimax operation Theorem 7.4.2 for the case $\mathcal{A} = \mathcal{O} = \mathbb{B}$.

The Two Hard Parts of AIXI

The mixture environment ξ uses the universal prior $2^{-K(\nu)}$ (where K is Kolmogorov complexity), which is uncomputable, and sums over the class $\mathcal{M}$ of *all* semicomputable chronological semimeasures, which is clearly intractable. The AIXI agent decomposes roughly into two parts, both problematic to evaluate directly.

- **Learning:** computing the Bayesian mixture ξ (based on the history so far), through which the agent implicitly updates its belief about which environment in the model class best represents the true environment μ .
- **Searching:** using ξ as a current best estimate of μ to look forward, considering the actions the agent could choose, the percepts the environment would respond with, the counter-actions to each percept, and so on up to the planning horizon m . Then the action that maximizes expected cumulative reward is chosen.

To compute each of the maximums in the expectimax, we must search through all actions in $\mathcal{A}$ and all percepts in $\mathcal{E}$, requiring $O(|\mathcal{A} \times \mathcal{E}|^m)$ time – intractable for even moderate m . We require efficiently computable approximations for *both* learning and searching to get a practically useful approximation of the AIXI agent – this is exactly what MCTS (searching) and CTW (learning) provide.

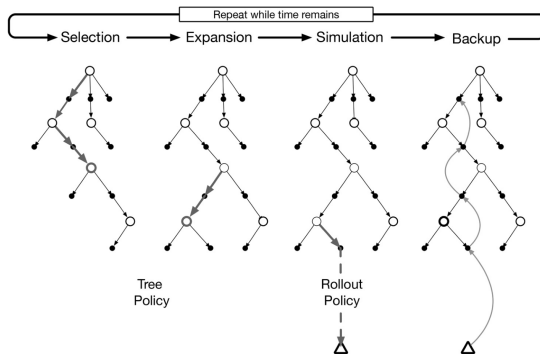
12.2.1 What Is Monte Carlo Tree Search?

- Instead of computing the expectimax sums explicitly, we *approximate* them by **sampling**: Monte Carlo Tree Search (MCTS) randomly explores paths of the search tree to estimate the expected value of different histories, rather than brute-forcing the whole tree.
- MCTS is an *anytime* search algorithm (it can be terminated at any point in time with a well-defined return value) that has been very successful in complex domains such as Go and Backgammon.
- **Chess analogy.** Chess has a branching factor of around 35 – planning 15 moves ahead naïvely requires exploring $35^{15} \approx 10^{23}$ nodes. A grandmaster does not search this space exhaustively; instead their intuition tells them which moves are worth considering and which can be discarded immediately (e.g. sacrificing the queen for a pawn is almost never worth exploring further).
- MCTS mimics this: from a given position, it decides which actions are worth exploring, and simulates a potential future using a model of the environment – much like a grandmaster mentally “simulates” an opponent when planning ahead.
- MCTS is especially effective in stochastic games with large branching factors (backgammon, poker), where sampled dice rolls / card deals quickly give a good estimate of an action’s value, and it can easily be parallelized over many rollouts.

The Four Steps of MCTS

We denote the action-observation search tree as Ψ (Psi). MCTS repeats four steps while execution time remains:

- 1 **Selection:** from the root, Ψ is traversed to a leaf chance node following a *tree policy*, balancing exploit (known-good moves) vs. explore.
- 2 **Expansion:** from the leaf chance node, a new unexplored action is selected and added as a child action node. Only one node is added per iteration.
- 3 **Simulation:** from the new action node, a possible outcome is simulated using a *rollout policy* (a cheap approximate behavior model) and the environment (or the agent's model of it), up to horizon m .
- 4 **Backup:** rewards from the simulated trajectory update the value/Q-value statistics of every node along the path traversed through Ψ



Choosing the Final Action

- When execution time has elapsed, MCTS chooses the next action, represented as a child of the root node, based on the estimated effect of taking that action. The root still represents the actual present state of the agent–environment interaction.
- Usually the action with the highest Q-value estimate is taken, but alternatively one could select randomly with a bias towards high-Q actions, or towards actions visited most often during backup. A Q-value estimate built from many rollouts should be more accurate than one built from few, so focusing purely on the current maximum may mean exploiting before enough exploration has happened.
- This is the action the agent chooses for interaction with the true environment. The environment then replies with a percept e ; MCTS restarts with the updated history $h := hae$ as the new root, and the tree can simply be “walked down” to the node associated with hae , reusing the existing statistics.

12.2.2 MCTS Algorithm: Unspecified Components

The generic description above leaves several components unspecified, later instantiated concretely for MC-AIXI-CTW:

- What statistics are stored in the nodes of Ψ .
- The choice of *tree policy* (the interaction between `SELECTACTION` and `SIMULATEACTION`).
- The new action during expansion, and the choice of *rollout policy* (both inside `EVALUATE`).
- Updating the node statistics (`UPDATEVALUE`).
- Once time has elapsed, choosing the best action from the accumulated statistics (`BESTACTION`).

Monte Carlo planning uses a tree Ψ that stores an estimate $\hat{V}(h)$ at each action node and $\hat{Q}(h, a)$ at each chance node. It initializes Ψ rooted at the current history h , and repeatedly calls `SAMPLE` until a time threshold $t_{\max}$ is exceeded (or a maximum number of simulations is used instead). Each call to `SAMPLE` generates a simulated sequence of actions (`SELECTACTION`) and percepts (`SIMULATEENV`). Once time is exceeded, `BESTACTION` picks the best action based on the statistics of Ψ . Through repeated simulated interaction, the agent acquires a better estimate of the Q-values of each history-action pair.

Algorithm 12.1: Monte Carlo Planning

Algorithm 12.1 Monte Carlo planning($h, m, t_{\max}$) [KS06]

Input: History h ; Horizon m ; Timeout time $t_{\max}$

Output: An action a

- 1: $\Psi :=$ action-observation tree starting at h
- 2: **while** running time less than $t_{\max}$ **do**
- 3: Sample(Ψ, h, m)
- 4: **return** BestAction(Ψ, h)

Sample (Algorithm 12.2) computes a recursive function taking as input the tree Ψ , the current history h , and the planning horizon m . The recursion ends when m reaches 0, at which point EVALUATE estimates the value of the (non-terminal) history. Otherwise, SAMPLE generates a new action a via SELECTACTION, and a new percept (o, r) given a via SIMULATEENV; it recursively calls itself on $h' := haor$ with horizon $m - 1$, adds the resulting value to reward r , updates the Q-value of (h, a) via UPDATEVALUE, and returns the estimate.

Algorithms 12.2–12.3: Sample and UpdateValue

Algorithm 12.2 Sample(Ψ, h, m)

Input: Tree Ψ ; history h ; horizon m

Output: Q-value estimate q

```
1: if  $m = 0$  then
2:   return 0
3: else
4:    $a := \text{SelectAction}(\Psi, h, m)$ 
5:    $(o, r) := \text{SimulateEnv}(\Psi, h, a)$ 
6:    $h' := haor$ 
7:    $q := r + \text{Sample}(\Psi, h', m - 1)$ 
8:   UpdateValue( $\Psi, h, a, q$ )
9: return  $q$ 
```

Algorithm 12.3 UpdateValue(Ψ, h, a, q)

Effect: update Q-value estimate on node ha

```
1:  $\Psi(ha).Q := \frac{q + \Psi(ha).N \times \Psi(ha).Q}{\Psi(ha).N + 1}$ 
2:  $\Psi(ha).N := \Psi(ha).N + 1$ 
```

Plain-English summary. q is the reward-to-go estimate that a single simulated rollout produced; N counts how many rollouts have passed through node ha ; Q is a running *average* of all q 's seen so far, updated incrementally so we never need to store the individual samples.

How Well Does Sample Perform?

The performance of `SAMPLE` depends on two things:

- **Accuracy of `SimulateEnv`.** For known environments, we sample $e \sim \mu(\cdot|ha)$ directly. For unknown environments (an agent exploring a maze, or playing a game against an opponent whose strategy is unknown) the agent needs a model of the environment to sample from – performance is then tied to how good that model is. Many MCTS variants sample from an internal model and then update it once a true percept is received.
- **The `SelectAction` function (also called the rollout policy).** The agent obviously cannot use MCTS itself to choose the rollout action (that would cause infinite regress), so we need a cheap, sensible rollout policy: an inexpensive estimate of what action the agent would take. In vanilla Monte Carlo planning, the rollout policy just picks uniformly at random. The *greedy* approach picks the action currently believed best, but can lead to suboptimal play if the current Q-value estimates are inaccurate. More advanced techniques learn a model (e.g. a neural network) of the agent's own behavior, and sample the rollout policy from that model.

UpdateValue is called whenever a recursive `SAMPLE` call returns a value; it combines the new Q-value sample with the statistics already collected at the chance node ha , and has no return value (only a side effect on the tree).

12.2.3 Bandits: A Simpler Sub-problem

Before explaining UCT, we motivate it with *bandit* problems, a simpler subclass of Markov Decision Processes that still have exploration–exploitation trade-offs. The name comes from “one-armed bandit”, a slot machine.

Definition 12.2.1 (Bandit)

Given a set of actions $\mathcal{A}$ and rewards $\mathcal{R}$, a *bandit* environment is a stochastic function $\mu : \mathcal{A} \rightarrow \Delta\mathcal{R}$.

Plain-English summary. $\Delta\mathcal{R}$ denotes the set of probability distributions over the reward set $\mathcal{R}$; so a bandit just maps each action to a (possibly random) reward, with no notion of state or observation – the agent is always “in the same situation”. Equivalently, a bandit is an MDP with a single state $\mathcal{S} = \{s\}$. The interaction history is $a_1r_1a_2r_2a_3r_3 \dots a_mr_m$ up to length m . The agent wants to maximize the sum of rewards over a fixed horizon. Performance is often measured by *regret*: the gap between the expected reward the agent obtains and that of the optimal policy π^* (or the best policy from a reference class Π). Despite the apparent simplicity, bandits are still non-trivial: the expected value of each action is unknown, so the agent must trade off exploiting the best-known action against exploring to improve its estimates.

Estimating Action Values, and ε -Greedy

If the agent knew the true expected reward $q^*(a) := \mathbb{E}_\mu[r_t | a_t = a]$ for every action, it could just always pick $a^* := \arg \max_a q^*(a)$. Instead, it estimates $\hat{Q}_t(a) \approx q^*(a)$ from the rewards obtained so far:

$$\hat{Q}_t(a) = \frac{\sum_{i=1}^{t-1} r_i \mathbb{I}[a_i = a]}{\sum_{i=1}^{t-1} \mathbb{I}[a_i = a]}$$

Plain-English summary. $\mathbb{I}[a_i = a]$ is 1 if action a was chosen at step i and 0 otherwise, so $\hat{Q}_t(a)$ is simply the average reward observed so far whenever action a was played. An agent playing *greedily* w.r.t. $\hat{Q}_t$ (always taking $\arg \max_a \hat{Q}_t(a)$) may never learn to abandon a suboptimal action that merely *looked* good early on – it stops collecting information once it believes it knows the best action.

$$a_t^{\varepsilon\text{-greedy}} = \begin{cases} \text{uniformly random } a \in \mathcal{A} & \text{probability } \varepsilon \\ \arg \max_{a \in \mathcal{A}} \hat{Q}_t(a) & \text{probability } 1 - \varepsilon \end{cases}$$

Plain-English summary. ε (epsilon) is a fixed exploration probability: with probability ε the agent explores completely at random, and otherwise exploits the current best-known action. Eventually the agent chooses the best action with probability $1 - \frac{|\mathcal{A}|-1}{|\mathcal{A}|}\varepsilon$. Too small an ε means slow learning; too large means the agent keeps choosing known-bad actions. ε -greedy is unsophisticated: it explores uniformly, potentially wasting time on actions already known to be poor.

Upper Confidence Bound (UCB)

A more sophisticated method focuses exploration on actions whose current value estimate $\hat{Q}_t(a)$ is *uncertain*.

UCB Q-value (Eq. 12.2.2)

$$Q_t^{\text{UCB}}(a) := \begin{cases} \hat{Q}_t(a) + c \sqrt{\frac{\ln(t)}{N_t(a)}} & N_t(a) > 0 \\ \infty & N_t(a) = 0 \end{cases}$$

Plain-English summary. $N_t(a) := \sum_{i=1}^{t-1} \mathbb{I}[a_i = a]$ counts how many times a has been chosen before step t ; $c > 0$ is a hyperparameter controlling how strong the exploration bonus is. The agent then plays $a_t^{\text{UCB}} := \arg \max_a Q_t^{\text{UCB}}(a)$. Intuitively, if action a has been tried $N_t(a)$ times, the relative error $\frac{Q_t(a) - \hat{Q}_t(a)}{Q_t(a)}$ is proportional to $\frac{1}{\sqrt{N_t(a)}}$; adding a bonus of this size encourages visiting actions that are either genuinely good, or whose value is still uncertain. Assigning a value of ∞ when $N_t(a) = 0$ both avoids a divide-by-zero *and* forces the agent to try every action at least once before anything else.

Python: ϵ -Greedy vs. UCB on a Toy Bandit

```
import numpy as np
rng = np.random.default_rng(0)

# Two-armed bandit: a0 gives reward 1 (always, mean 1);
# a1 gives reward 5 w.p. 0.3, else 0 (mean 1.5, the better arm)
def reward(a):
    return 1.0 if a == 0 else (5.0 if rng.random() < 0.3 else 0.0)

def run(policy, T=200, eps=0.2, c=2.0):
    N = np.zeros(2); Q = np.zeros(2)
    for t in range(1, T + 1):
        if policy == "eps-greedy":
            a = rng.integers(2) if rng.random() < eps else int(np.argmax(Q))
        else: # UCB
            score = [np.inf if N[i] == 0 else
                    Q[i] + c * np.sqrt(np.log(t) / N[i]) for i in range(2)]
            a = int(np.argmax(score))
        r = reward(a)
        N[a] += 1; Q[a] += (r - Q[a]) / N[a]
    return Q, N

for policy in ["eps-greedy", "UCB"]:
    Q, N = run(policy)
    print(f"{policy:11s} Q-hat={np.round(Q,2)} N={N}")
# eps-greedy Q-hat=[1.02 1.62] N=[ 90 110] (varies run to run)
# UCB Q-hat=[1.0 1.58] N=[ 46 154] (UCB commits to the better arm faster)
```

12.2.4 The UCT Algorithm

Upper Confidence Trees (UCT) extends Monte Carlo planning (Algorithm 12.1) by replacing uniform action selection with a modified UCB rule.

- **Random** action selection is inefficient: it wastes computation exploring actions that are obviously bad a priori (e.g. moving a chess piece into certain capture).
- **Greedy** action selection exploits too early, before adequate statistics are collected, getting stuck with overconfident value estimates.

UCT balances the two: the UCT-value has an exploration bonus proportional to $\frac{1}{\sqrt{N}}$, where N is the number of times the action has been sampled. If an action has seldom been taken, its Q-value estimate is uncertain, so the bonus is large, encouraging the agent to explore it.

Definitions 12.2.3–12.2.4 (Visit count & UCT action selection)

The *visit count* $N(h)$ of history h is the number of times h has been sampled by UCT; $N(ha)$ similarly counts visits to the history-action pair. The action chosen by UCT is

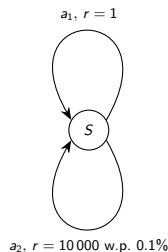
$$a_{UCT}(h) := \arg \max_{a \in \mathcal{A}} Q_{UCT}(h, a), \quad Q_{UCT}(h, a) := \begin{cases} \frac{1}{m} \hat{Q}(h, a) + C \sqrt{\frac{\ln N(h)}{N(ha)}} & N(ha) > 0 \\ \infty & N(ha) = 0 \end{cases}$$

Interpreting the UCT Formula

- $\hat{Q}(h, a)$ is the current unbiased frequentist estimate (Algorithm 12.3) of the Q-value of history h , action a ; dividing by m (the planning horizon) rescales it to be comparable to the bonus term, since rewards accumulate over up to m steps.
- $C \in \mathbb{R}$ is a positive exploration–exploitation hyperparameter. Small C makes the agent focus on exploiting its current best-looking actions, producing a deep but narrow tree; large C incentivizes exploration, producing a shallow but broad tree.
- Assigning ∞ when $N(ha) = 0$ forces UCT to always prioritize a never-before- chosen action a over anything else, exactly as with plain UCB.
- The logarithmic term $\ln N(h)$ in the exploration bonus is crucial: it ensures the agent chooses *every* action infinitely often (even ones with low current Q-value), though at a logarithmically decreasing rate. This is what guarantees convergence, both in theory and in practice.

The `SELECTACTION` part of MCTS matters because we don't want the search tree to go down paths that give low reward, but we also want to explore history-action pairs we've rarely visited – exactly the same exploration/exploitation trade-off as in the bandit case, now applied recursively at every node of the tree.

Example 12.2.5: Upper Confidence Bounds for a Bandit



Consider this (trivially) Markov decision process, technically a bandit: a single state S , both actions lead back to S , and the history is irrelevant, so h is always empty. Action a_1 *always* returns reward 1. Action a_2 returns reward 10 000 with probability 10^{-3} , else 0 – an *expected* return of $r = 10$. Hence the optimal policy is to *always* choose a_2 .

Now suppose the agent follows a UCT-like rule but with the log term **removed**:

$$a_{UCT'}(h) := \arg \max_a Q_{UCB'}(h, a), \quad Q_{UCB'}(h, a) := \begin{cases} \frac{1}{m} \hat{Q}(h, a) + C \sqrt{\frac{1}{N(ha)}} & N(ha) > 0 \\ \infty & N(ha) = 0 \end{cases}$$

which is the same as a_{UCT} but without $\ln N(h)$.

Example 12.2.5 (continued): Why $\ln N(h)$ Matters

Unless the agent got particularly lucky and observed $r = 10\,000$ from a_2 soon, it will choose a_2 only finitely many times and then always choose a_1 thereafter. Assume $\hat{Q}(h, a_1) = 1$ (its true value) and, for a moment, $\hat{Q}(h, a_2) = 0$ (as a_2 has not yet returned a non-zero reward). Then:

$$Q_{UCB'}(h, a_1) = \frac{\hat{Q}(h, a_1)}{m} + C\sqrt{\frac{1}{N(ha_1)}} > \frac{1}{m}, \quad Q_{UCB'}(h, a_2) = \frac{\hat{Q}(h, a_2)}{m} + C\sqrt{\frac{1}{N(ha_2)}} = C\sqrt{\frac{1}{N(ha_2)}}$$

So once $N(a_2) > (Cm)^2$, we get $\frac{1}{m} > C\sqrt{1/N(ha_2)}$ and the agent **never again** chooses a_2 – it never learns the optimal policy!

For $C = 2$, $m = 10$: we need $N(a_2) > 400$, and there is a $\approx (1 - 0.001)^{400} \approx 67\%$ chance a_2 never returns a reward over 400 interactions.

Adversarial construction. Fix $\delta > 0$. Choose $\varepsilon = 1 - \exp(-\frac{\ln \delta}{C^2 m^2})$ and add an action returning reward $10/\varepsilon$ with probability ε (expected value still 10). Then

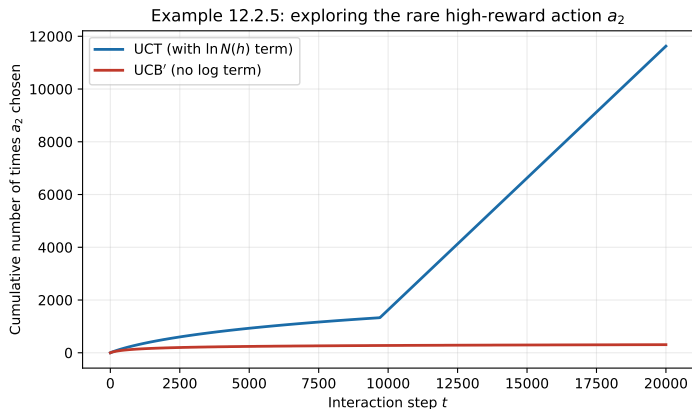
$$\mathbb{P}(a_2 \text{ finitely often}) \geq \mathbb{P}(a_2 \text{ returns 0 over } C^2 m^2 \text{ trials}) = (1 - \varepsilon)^{C^2 m^2} = \delta.$$

So for *any* fixed horizon m and exploration constant C , we can build an adversarial bandit where $a_{UCT'}$ fails to find the optimal action with probability at least δ – no matter how small δ is. This is exactly the failure mode the $\ln N(h)$ term is designed to prevent.

Why the Logarithmic Bonus Is the Right Fix

- One could try other exploration schedules – a decaying ε_t -greedy, or a *Boltzmann distribution* (sampling a with probability $\propto \exp(\hat{Q}(h, a)/T)$, with “temperature” T controlling how sharp the distribution is, analogous to statistical physics: a very cold system, $T \rightarrow 0$, barely moves, while a hot one, $T \rightarrow \infty$, wiggles a lot). But these all have their own problems: decaying schedules are hard to tune for arbitrary environments, and choosing the right cooling rate for T is itself an unsolved sub-problem.
- The universal solution is to add $\ln N(h)$ to the numerator of the exploration bonus. This guarantees every action is explored at least logarithmically often (hence infinitely often over an infinite horizon), so the agent eventually samples a_2 enough times to learn it is the better action.
- One could use other functions in $o(\sqrt{n})$, such as $\ln^2 N(h)$ or $\sqrt[3]{N(h)}$, but it can be shown that $\ln N(h)$ is *optimal* in the sense of giving the best bound on the *worst-case regret* – the largest possible gap between the expected reward of π_{UCT} and that of the optimal policy π_μ^* , over any choice of environment μ .

Illustration: UCT vs. UCB' on Example 12.2.5



A simulation of both rules on Example 12.2.5's bandit ($C = 2$, $m = 10$, 20 000 steps). **UCT** (blue, with the $\ln N(h)$ term) keeps sampling the rare high-reward action a_2 throughout the run. **UCB'** (red, without the log term) samples a_2 a bounded number of times and then stops exploring it altogether, exactly as predicted by the argument on the previous slide.

Python: Reproducing Example 12.2.5 Numerically

```
import numpy as np
rng = np.random.default_rng(1)

def simulate(use_log, T=20000, C=2.0, m=10):
    N = np.zeros(2); Q = np.zeros(2)
    picks_a2 = 0
    for t in range(1, T + 1):
        scores = np.empty(2)
        for a in range(2):
            if N[a] == 0:
                scores[a] = np.inf
            else:
                bonus = C * np.sqrt((np.log(t) if use_log else 1.0) / N[a])
                scores[a] = Q[a] / m + bonus
        a = int(np.argmax(scores))
        r = 1.0 if a == 0 else (10000.0 if rng.random() < 1e-3 else 0.0)
        N[a] += 1; Q[a] += (r - Q[a]) / N[a]
        picks_a2 += (a == 1)
    return N[1], picks_a2 # final visit count of a2, total times a2 was picked

for use_log, name in [(False, "UCB' (no log)"), (True, "UCT (with log)"]:  
    N_a2, picks = simulate(use_log)  
    print(f"{name:16s}: N(a2) after 20000 steps = {int(N_a2):5d}")  
# UCB' (no log) : N(a2) after 20000 steps ~ 30 (gets stuck avoiding a2 early on)  
# UCT (with log) : N(a2) after 20000 steps ~ 3000+ (keeps exploring a2 throughout)
```

This matches Figure 12.9's illustration above: UCT keeps sampling the rare high-reward action a_2

Convergence of UCT

A general theoretical result [KS06] shows that the estimate of the expectimax value produced by UCT, $V_{UCT}(h) := V^{\pi_{UCT}}(h)$, **converges in probability** to the optimal expectimax value $V^*(h)$, with respect to the implied distribution $\mu^{\pi_{UCT}}$, where μ is any finite-horizon MDP.

This shows UCT is indeed a good choice for estimating the value function, as the estimate converges to the true value. It does *not*, however, say how fast it converges: [KS06] also prove a convergence-rate result, but with an exponentially large constant. In practice, convergence is often fast enough that UCT can be used effectively.

12.2.5 The ρ UCT Algorithm

ρ **UCT** (ρ pronounced “rho”; Algorithm 12.5) extends UCT to approximate the expectimax tree when the true environment μ is *unknown* and replaced by an estimate ρ (rho). For ρ UCT, ρ takes the place of the `SIMULATEENV` function and can be queried directly for samples. The action-selection criterion is the same UCB-based rule as UCT (Definition 12.2.4).

For ρ UCT, the tree Ψ stores at action nodes an estimate $\hat{V}(h)$ of the value, and at chance nodes an estimate $\hat{Q}(h, a)$ of the Q-value; all nodes also track visit counts $N(h)$ or $N(h, a)$. Both estimates are *averages* of all the samples of value/Q-value obtained from rollouts. Given a current estimate $\hat{Q}_N := \frac{1}{N}(q_1 + \dots + q_N)$ after N visits and a new sample q_{N+1} , we update efficiently:

$$\hat{Q}_{N+1} := \frac{N\hat{Q}_N + q_{N+1}}{N+1}$$

and increment $N := N + 1$. (This is the same incremental-average update as Algorithm 12.3.)

Theorem 12.2.6: Consistency of ρ UCT

Theorem 12.2.6 (Consistency of ρ UCT [VNHS10, KS06])

Let μ be the true underlying environment. For all $\varepsilon > 0$, for all histories h ,

$$\lim_{N(h) \rightarrow \infty} \mathbb{P}\left(|V_{\mu}^{\pi_{\rho UCT}}(h) - \hat{V}_{\mu}^{\pi_{\rho UCT}}(h)| \leq \varepsilon\right) = 1$$

where $\pi_{\rho UCT}$ is the policy that follows the ρ UCT algorithm.

Plain-English summary. As the number of visits $N(h)$ to history h grows without bound, the probability that ρ UCT's *estimated* value $\hat{V}_{\mu}^{\pi_{\rho UCT}}(h)$ is within any fixed tolerance ε of the *true* value $V_{\mu}^{\pi_{\rho UCT}}(h)$ tends to 1. In other words, given enough rollouts, ρ UCT's value estimates become arbitrarily accurate (with probability approaching 1), so the optimal agent using the approximate value function $\hat{V}_{\rho}^*$ will behave like the true optimal agent derived from V_{ρ}^* , at least in the limit. This theorem gives no indication of the *rate* of convergence, only that convergence happens eventually with probability 1.

Algorithms 12.4–12.5: The Agent Loop and ρ UCT I

Algorithm 12.4 Agent-Environment Interaction Loop

Input: Horizon m ; timeout $t_{\max}$; rollout policy π ; true environment μ ; environment model ρ

Effect: generates interaction history h

- 1: $h := \epsilon$
- 2: **while** True **do**
- 3: $a := \rho\text{UCT}(h, m, t_{\max}, \pi, \rho)$
- 4: $e \sim \mu(ha)$
- 5: $h := hae$

Algorithm 12.5 $\rho\text{UCT}(h, m, t_{\max}, \pi, \rho)$

Input: History h ; horizon m ; timeout $t_{\max}$; rollout policy π ; environment model ρ

Output: Action a chosen by ρUCT

- 1: $\Psi := \text{action-obs. tree, root } h$
- 2: **while** running time $< t_{\max}$ **do**
- 3: $\text{MCTS}(\Psi, h, m, \pi, \rho)$
- 4: **return** $\text{BestAction}(\Psi, h)$

Note the true environment μ only appears in the outer interaction loop (Algorithm 12.4), sampling the *real* percept once an action is taken; *inside* the search (Algorithm 12.5), only the model ρ is used to simulate hypothetical futures.

Algorithms 12.4–12.5: The Agent Loop and ρ UCT II

Algorithm 12.6 MCTS(Ψ, h, m, π, ρ)

Output: Reward sum R

```
1: if  $m = 0$  then                                ▷ end of horizon
2:   return 0
3: else if  $N(h) = 0$  then                          ▷ reached leaf
4:    $R := \text{Rollout}(h, m, \pi, \rho)$                 ▷ simulation
5: else
6:    $a := \text{SelectAction}(\Psi, h, C)$                 ▷ continue
7:    $R := \text{SampleObservations}(\Psi, h, a, m, \rho)$ 
8:    $\hat{V}(h) := \frac{1}{N(h)+1} [R + N(h)\hat{V}(h)]$ 
9:    $N(h) := N(h) + 1$ 
10: return  $R$ 
```

Algorithm 12.7

SampleObservations(Ψ, h, a, m, ρ)

Output: Reward sum R

```
1: Sample  $e \sim \rho(\cdot | ha)$ 
2: if  $N(ha) = 0$  then
3:   Create node  $\Psi(hae)$  as child of  $\Psi(ha)$ 
4:  $h' := hae$ 
5:  $R := r + \text{SampleActions}(\Psi, h', m - 1, \pi, \rho)$ 
6:  $\hat{Q}(ha) := \frac{1}{N(ha)+1} [\text{reward} + N(ha)\hat{Q}(ha)]$ 
7:  $N(ha) := N(ha) + 1$ 
8: return  $R$ 
```

Algorithms 12.8–12.9: SelectAction and Rollout

Algorithm 12.8 SelectAction(Ψ, h, C)

Input: search tree Ψ ; history h ; exploration constant C

Output: Action a

- 1: $\hat{\mathcal{A}} := \{a \in \mathcal{A} : N(ha) = 0\}$
- 2: **if** $\hat{\mathcal{A}} \neq \{\}$ **then**
- 3: Pick $a \in \hat{\mathcal{A}}$ uniformly at random
- 4: Create node $\Psi(ha)$
- 5: **else**
- 6: $a := \arg \max_{a \in \mathcal{A}} \left\{ \frac{1}{m} \hat{V}(ha) + C \sqrt{\frac{\ln(N(h))}{N(ha)}} \right\}$
- 7: **return** a

Algorithm 12.9 Rollout(h, m, π, ρ)

Output: Reward sum R

- 1: $R := 0$
- 2: **for** $i := 1$ to m **do**
- 3: Sample a from $\pi(\cdot|h)$
- 4: Sample e from $\rho(\cdot|ha)$
- 5: $R := R + r$
- 6: $h := hae$
- 7: **return** R

Plain-English summary. SELECTACTION always tries a never-visited action first (forcing exploration of the whole action set), otherwise applies the UCT rule with constant C . ROLLOUT is the cheap simulation used once we fall off the edge of the tree: it repeatedly samples an action from the rollout policy π and a percept from the model ρ , accumulating reward, until the horizon m is exhausted.

12.2.6 Parallelization of MCTS

MCTS can be performed in parallel in (at least) three ways [CWvdH08a]:

- **Leaf Parallelization.** Selection and expansion happen sequentially as normal. Instead of a single rollout during simulation, many independent rollouts are performed *in parallel*, and their statistics aggregated before a sequential backpropagation. Since each rollout is independently generated, no synchronization of shared resources is required.
- **Root Parallelization.** Multiple trees are instantiated, and MCTS is run on each independently. Once the allotted time expires, the statistics of the children of each root are merged into a single tree, which is then used to select an action.
- **Tree Parallelization.** A single tree is shared between multiple workers, all running MCTS simultaneously. Synchronization mechanisms prevent two workers accessing the same node at once: either a *global mutex* (only one worker updates statistics at a time, while others do rollouts) or *local mutexes* (workers lock a node only while updating it, then move on). A *mutex* (“mutual exclusion”) restricts a shared resource to one worker at a time; local mutexes waste less time locking/unlocking but add overhead of their own.

Experiments [CWvdH08a, MPVDHV15] indicate both tree and root parallelization are promising; leaf parallelization performs poorly by comparison, even though far more games are evaluated per second – suggesting agent performance depends more on the *number of distinct nodes explored* than on the raw accuracy of rollout statistics.

12.2.7 Episodic Environments

So far we assumed interaction never terminates, and the agent maximizes reward over a forward moving horizon. *Episodic* environments instead split the interaction into disjoint *episodes*: each episode doesn't affect the others, and when an episode terminates the environment resets to its original configuration.

- We can encode episodic environments as a special case of non-episodic ones, by adding an extra observation o_{end} signalling termination: when issued, the interaction history is deleted and interaction starts afresh.
- This enables optimizations that were not otherwise possible. For example, chess is episodic (the game terminates on loss/win/draw, and history from one game is irrelevant to subsequent games), so we can reuse statistics from previous games whenever the same initial board position recurs.
- In many episodic environments, reward is only given at the very end of the episode (e.g. chess naturally issues $-1/0/1$ for loss/draw/win, and 0 for every state in progress).
- MCTS changes accordingly: previously we ran the four steps until the time budget for choosing one move was exhausted; the agent selects an action a , μ returns a percept e , and $h' := hae$; MCTS restarts at h' , discarding the other children of the tree.
- For episodic environments, instead we can reuse large parts of the tree: once an episode terminates, we move back to the root of the search tree and continue from there, retaining the statistics accumulated from previous episodes – letting the agent learn from mistakes made in earlier episodes.

From MCTS to a Learned Model

We now have an efficient approximation of *searching* (ρ UCT) for a given, fixed environment ρ . But the true environment μ is unknown, so we wish to instantiate ρ UCT with a **Bayesian mixture** ξ (χ_i) over environments that assigns non-zero weight to the true environment μ . AIXI uses ξ as its best estimate of the true environment, and updates ξ based on the percepts received. We approximate ξ using the **Context Tree Weighting (CTW)** algorithm (recall Section 4.5).

We first modify CTW to work on arbitrary (finite) action spaces rather than binary strings. We do this so as to preserve the *type* information of the percepts received – so that, at the end of the day, the agent only receives bits, but we wish to distinguish the observation bits from the reward bits.

12.3.1 Action-Conditional CTW

Recall that CTW is an *online prediction* algorithm (giving a new prediction after every bit observed, as opposed to an *offline* algorithm that first sees all the data) which efficiently computes the probability

CTW Mixture (recap)

$$P_D^{\text{CTW}}(x_{1:t}) \equiv P_{\mathcal{T}_D}(x_{1:t}) = \sum_{S \in \mathcal{C}_D} w_S P_{S, \text{KT}}(x_{1:t})$$

- $x_{1:t}$ is a binary sequence.
- S (calligraphic S) is a *suffix set* (Definition 4.3.3) – a way of partitioning histories by their most recent bits – or equivalently a *suffix tree* (Definition 4.3.6).
- $w_S := 2^{-\Gamma_D(S)}$ is the complexity prior on S , defined via the tree *model code length* $\Gamma_D(S)$ (Definition 4.3.20) – simpler suffix sets get a larger prior weight, per Occam's razor.
- $\mathcal{C}_D$ is the set of all suffix sets of depth at most D .
- $P_{S, \text{KT}}$ is the PST-KT probability associated with S (Definition 4.3.24) – the probability assigned by a prediction suffix tree using the KT estimator at each leaf.

CTW computes this probability in time $O(D)$ (Remark 4.5.3), compared to the naïve sum over all suffix sets which takes $O(2^{2^D})$ time – a double-exponential speedup.

Extending CTW to Action-Conditional Prediction

The idea of CTW was extended in [VNHS10] from predicting bits to predicting bits *conditioned on actions*. Instead of estimating $P(x_{1:t})$, the extension estimates $\mu(e_{1:t} \| a_{1:t})$ (recall the $\|$ notation of Definition 6.3.1, meaning “interleaved with”):

Action-Conditional CTW (Eq. 12.3.1)

$$P_D^{\text{CTW}}(e_{1:t} \| a_{1:t}) = \sum_{S \in \mathcal{C}_D} w_S P_{S, \text{KT}}(e_{1:t} \| a_{1:t})$$

where $P_{S, \text{KT}}$ is extended from Definition 4.3.24 to give a conditional probability distribution over percepts conditioned on histories.

KT estimator (recap, Lemma 4.1.2)

$$P_{kt}(x_{1:t}) := P_{kt}(x_t | x_{<t}) P_{kt}(x_{<t}), \quad P_{kt}(\epsilon) := 1$$
$$P_{kt}(x_{t+1}=0 | x_{1:t}) = \frac{a + 1/2}{a + b + 1}, \quad P_{kt}(x_{t+1}=1 | x_{1:t}) = 1 - P_{kt}(0 | x_{1:t}) = \frac{b + 1/2}{a + b + 1}$$

Plain-English summary. a, b are the counts of zeros/ones in $x_{1:t}$ so far; the KT estimator is a simple, closed-form Bayesian estimate of the next bit's probability from these counts – the building block CTW mixes over many contexts.

12.3.2 Action-Conditional PST (AC-PST)

Percept and action sequences are usually *non-binary* (more than two actions or percepts). The solution: encode actions and percepts as binary strings, use this encoding to turn a history into a bit sequence, and perform prediction on the result. This generalizes PSTs (Section 4.3.1) to non-binary alphabets, giving *action-conditional prediction suffix trees (AC-PST)*.

- Assume w.l.o.g. that $|\mathcal{A}| = 2^{\ell_{\mathcal{A}}}$ and $|\mathcal{E}| = 2^{\ell_{\mathcal{E}}}$ for some $\ell_{\mathcal{A}}, \ell_{\mathcal{E}} > 0$ (padding with unreachable dummy actions/percepts if needed).
- Enumerate all $2^{\ell_{\mathcal{A}}}$ actions, and assign each a bit sequence of length $\ell_{\mathcal{A}}$:
 $\llbracket a \rrbracket \equiv a[1:\ell_{\mathcal{A}}] := a[1]a[2] \dots a[\ell_{\mathcal{A}}] \in \mathbb{B}^{\ell_{\mathcal{A}}}$. Similarly encode percepts e and sequences $\llbracket a_{1:t} \rrbracket := \llbracket a_1 \rrbracket \llbracket a_2 \rrbracket \dots \llbracket a_t \rrbracket$.
- An AC-PST is defined exactly like a PST (Definition 4.3.11) with a suffix set $\mathcal{S}$, parameters $\Theta_{\mathcal{S}}$, and tree $\Psi_{\mathcal{S}, \Theta_{\mathcal{S}}}$ – the difference is purely in *how updates are performed*.

Update procedure (after a warm-up period long enough for the encoded history $\llbracket h \rrbracket$ to reach the tree depth):

- 1 An action a is selected by the agent; $h := h\llbracket a \rrbracket$.
- 2 The environment responds with percept $\llbracket e \rrbracket$.
- 3 For $i = 1, \dots, \ell_{\mathcal{E}}$: (a) predict bit $e[i]$ using distribution $\theta_{\beta_{\mathcal{S}}(h)}$, without having seen it yet; (b) observe $e[i]$ and update $\theta_{\beta_{\mathcal{S}}(h)}$ via the KT estimator (Algorithm 4.3); (c) set $h := he[i]$.

Mixing over AC-PSTs the same way we mix over CTWs gives the *Action-Conditional CTW* method.

Example 12.3.2: Factored CTW Beats Plain CTW

Consider a trivial environment with $\mathcal{A} = \mathcal{O} = \mathcal{R} = \mathbb{B}$. The environment issues observation o_t uniformly at random, and reward $r_t = 1$ iff the agent's action a_t matched the *observation before the most recent one*, o_{t-1} :

$$\mu(o_t r_t | h_{<t} a_t) = \begin{cases} 1/2 & o_t r_t = (0, 1) \text{ and } a_t = o_{t-1} \\ 1/2 & o_t r_t = (1, 1) \text{ and } a_t = o_{t-1} \\ 1/2 & o_t r_t = (0, 0) \text{ and } a_t \neq o_{t-1} \\ 1/2 & o_t r_t = (1, 0) \text{ and } a_t \neq o_{t-1} \end{cases}$$

Optimal behavior is trivial: remember the last observation, and play it as the next action. We encode the percept $e_t = o_t r_t$ as a 2-bit code, first bit observation, last bit reward.

The problem. When the agent receives a percept, the CTW tree is updated one bit at a time. Suppose the history so far, as a raw bit stream, is $\dots 0101$ and the next bit to update is 0. It is *ambiguous* whether we are parsing the first or second bit of the percept:

- $\dots 0101 = \dots a_{t-1} o_{t-1} r_{t-1} \mathbf{a_t}$ and $0 = o_t$ (we are predicting an observation bit), **or**
- $\dots 0101 = \dots o_{t-1} r_{t-1} \mathbf{a_t} o_t$ and $0 = r_t$ (we are predicting a reward bit)!

Without knowing which bit is which, the agent cannot distinguish rewards from observations, might confuse a reward bit of 1 for the observation, and choose action 1 in response, leading to lower reward. *Bits alone are not sufficient; the agent needs to know what those bits represent.*

12.3.3 Factored Action-Conditional CTW (FAC-CTW)

To preserve *type* information (distinguishing bits of actions from bits of percepts, and different bit-positions within a percept), [VNHS10] introduced **Factored Action-Conditional CTW (FAC-CTW)**, which chains together one Action-Conditional PST for *each* of the $\ell_{\mathcal{E}}$ bits of the percept space.

- We keep $\ell_{\mathcal{E}}$ separate context trees $\mathcal{T}_{D_1}, \dots, \mathcal{T}_{D_{\ell_{\mathcal{E}}}}$ where tree $\mathcal{T}_{D_i}$ is only ever updated on bit $e[i]$ of a percept e , given the history h concatenated with the prior bits $e[1:i-1]$ of the *current* percept as extra context. Ensuring each tree is only ever updated on the same bit-position preserves type information.
- Each tree has depth $D_i := D + i - 1$, so tree $\mathcal{T}_{D_i}$ sees the same D bits of context from h as every other tree, *plus* $i - 1$ bits of context from the current percept, $e[1:i-1]$.
- In practice: given history h and new percept e , tree $\mathcal{T}_{D_1}$ is updated on $e[1]$ given context h ; tree $\mathcal{T}_{D_2}$ is updated on $e[2]$ given context $he[1]$; and so on up to $\mathcal{T}_{D_{\ell_{\mathcal{E}}}}$, updated on $e[\ell_{\mathcal{E}}]$ given context $he[1:\ell_{\mathcal{E}}-1]$. Each tree uses the ordinary CTW update rule (Algorithm 4.4).

FAC-CTW: the Formal Mixture (1/2)

Our model class is now a *family* of tree sources $\mathcal{C} := \mathcal{C}_{D_1} \times \mathcal{C}_{D_2} \times \cdots \times \mathcal{C}_{D_{\ell_{\mathcal{E}}}}$, where $\mathcal{C}_{D_i} = \mathcal{C}_{D_i}$ is the set of all suffix sets of depth at most D_i . We define an element of this family as $\mathcal{S} = (\mathcal{S}_1, \dots, \mathcal{S}_{\ell_{\mathcal{E}}})$; the trees are updated independently of each other, so

Factored likelihood (Eq. 12.3.3)

$$P_{\mathcal{S}}(e_{1:t} \| a_{1:t}) = \prod_{i=1}^t P_{\mathcal{S}}(e_i | a e_{<i} a_i) = \prod_{i=1}^t \prod_{j=1}^{\ell_{\mathcal{E}}} P_{\mathcal{S}_j}(e_i[j] \mid \llbracket h_{<i} \rrbracket e_i[1:j-1])$$

Plain-English summary. $e_{1:t}[j]$ denotes $e_1[j] \dots e_t[j]$, the j -th bit from every percept concatenated together, and $e_{1:t}[\setminus j]$ denotes $e_{1:t}$ with the j -th bit of every percept *omitted*. Each of the $\ell_{\mathcal{E}}$ trees predicts its own bit-position using the ordinary history *plus* the earlier bits of the current percept as extra context.

FAC-CTW: the Formal Mixture (2/2)

Assuming the priors over $\mathcal{C}_{D_1}, \dots, \mathcal{C}_{D_{\ell_{\mathcal{E}}}}$ are independent:

$$w_S := P(\mathcal{S}_1, \dots, \mathcal{S}_{\ell_{\mathcal{E}}}) = \prod_{j=1}^{\ell_{\mathcal{E}}} P(\mathcal{S}_j) = \prod_{j=1}^{\ell_{\mathcal{E}}} 2^{-\Gamma_{D_j}(\mathcal{S}_j)} = 2^{-\sum_{j=1}^{\ell_{\mathcal{E}}} \Gamma_{D_j}(\mathcal{S}_j)}$$

Letting $\mathcal{M} = \mathcal{C}_{D_1} \times \dots \times \mathcal{C}_{D_{\ell_{\mathcal{E}}}}$, the FAC-CTW mixture model is

FAC-CTW mixture (Eq. 12.3.4)

$$\xi(\mathbf{e}_{1:t} \parallel \mathbf{a}_{1:t}) := \sum_{S \in \mathcal{M}} w_S P_S(\mathbf{e}_{1:t} \parallel \mathbf{a}_{1:t})$$

Plain-English summary. w_S is the prior weight of the whole family $S = (\mathcal{S}_1, \dots, \mathcal{S}_{\ell_{\mathcal{E}}})$, which – because the per-bit priors are independent – is simply the *product* of each tree's individual complexity prior. ξ then sums the weighted likelihood over every possible combination of trees in the family $\mathcal{M}$.

FAC-CTW Factorizes into $\ell_{\mathcal{E}}$ Independent CTW Mixtures

Rearranging Eq. 12.3.4:

$$\begin{aligned}\xi(e_{1:t} \| a_{1:t}) &= \sum_{S \in \mathcal{M}} w_S P_S(e_{1:t} \| a_{1:t}) = \sum_{S_1 \in \mathcal{C}_{D_1}} \cdots \sum_{S_k \in \mathcal{C}_{D_k}} 2^{-\sum_{i=1}^k \Gamma_{D_i}(S_i)} \prod_{j=1}^k P_{S_j}(e_{1:t}[j] | a_{1:t}, e_{1:t}[\setminus j]) \\ &= \prod_{j=1}^k \left(\sum_{S_j \in \mathcal{C}_{D_j}} 2^{-\Gamma_{D_j}(S_j)} P_{S_j}(e_{1:t}[j] | a_{1:t}, e_{1:t}[\setminus j]) \right) = \prod_{j=1}^k P_{D_j}^{\text{CTW}}(e_{1:t}[j] | a_{1:t}, e_{1:t}[\setminus j])\end{aligned}$$

(writing $k := \ell_{\mathcal{E}}$.) Each factor $P_{D_j}^{\text{CTW}}(e_{1:t}[j] | a_{1:t}, e_{1:t}[\setminus j])$ is a conditional probability, computed the same way as for a single CTW tree (Eq. 4.5.5).

Plain-English summary. The big, seemingly intractable mixture ξ over the entire family of trees factors exactly into a *product* of $\ell_{\mathcal{E}}$ independent, ordinary CTW mixtures – one per percept bit-position. So we never actually have to enumerate the huge joint family $\mathcal{M}$: we maintain $\ell_{\mathcal{E}}$ separate (and much smaller) CTW trees, and multiply their root probabilities together to recover ξ .

Algorithms 12.10–12.11: (Factored) Action-Conditional CTW

Algorithm 12.10 Action-Conditional CTW

Require: Context tree $\mathcal{T}_D$

Input: action-percept stream $h = a_1 e_1 a_2 e_2 a_3 e_3 \dots$

Output: $P_D^{\text{CTW}}(e_{1:} \| a_{1:})$ bit by bit

Effect: maintain CTW tree $\mathcal{T}_D$

- 1: Interact with environment to generate initial history h
- 2: **while** True **do**
- 3: Agent chooses action $a \in \mathcal{A}$
- 4: $h := ha$
- 5: Send a to environment, receive e
- 6: **for** $1 \leq i \leq \ell_{\mathcal{E}}$ **do**
- 7: $p := \text{CTWPrediction}(\mathcal{T}_D, h)$
- 8: Use p to predict $e[i]$
- 9: Observe (and receive) next bit $e[i]$
- 10: CTWUpdate($\mathcal{T}_D, h, e[i]$)
- 11: Set $h := he[i]$

Algorithm 12.11 FAC-CTW

Input: Model class $\mathcal{M}$

Effect: maintain the family of CTW trees $\mathcal{T}_{D_1}, \dots, \mathcal{T}_{D_k}$

- 1: $h := \epsilon$
- 2: $t := 1$
- 3: Initialize $k := \ell_{\mathcal{E}}$ context trees $\mathcal{T}_{D_1}, \dots, \mathcal{T}_{D_k}$
- 4: **while** True **do**
- 5: Agent chooses action a_t
- 6: $h := ha_t$
- 7: Agent transmits a_t , receives e_t
- 8: **for** $1 \leq i \leq k$ **do**
- 9: CTWUpdate($\mathcal{T}_{D_i}, he[1, i-1], e_i$)
- 10: $h := he_t$
- 11: $t := t + 1$

Python: A Toy KT / Factored-CTW Demo

```
# One KT estimator per (bit-position, previous-action) context -- a depth-0
# "factored CTW" -- applied to Example 12.3.2: reward=1 iff a_t == o_{t-1}.
import numpy as np
rng = np.random.default_rng(3)

def kt_predict(counts):
    # counts = [#zeros, #ones] so far
    a, b = counts
    return (a + 0.5) / (a + b + 1) # P(next bit = 0)

def kt_update(counts, bit):
    counts[bit] += 1

# 2 contexts (prev action) x 2 bit-positions (obs, reward) = 4 KT trees,
# exactly like FAC-CTW's separate tree per percept bit.
kt = {(pos, ctx): [1, 1] for pos in (0, 1) for ctx in (0, 1)}

o_prev, correct = 0, 0
for t in range(2000):
    a_prev = o_prev # agent repeats last obs.
    o_t = rng.integers(2)
    r_t = 1 if a_prev == o_prev else 0 # definition: a_t == o_{t-1}
    for pos, bit in enumerate((o_t, r_t)):
        p0 = kt_predict(kt[(pos, a_prev)])
        correct += int((0 if p0 > 0.5 else 1) == bit)
        kt_update(kt[(pos, a_prev)], bit)
    o_prev = o_t

print(f"bit accuracy over 2000 steps x 2 bits: {correct/4000:.3f}")
# -> ~1.0 for the (deterministic) reward bit, ~0.5 for the (random) obs. bit
```

12.4 Combining ρ UCT and ξ_{FAC} : MC-AIXI-CTW

We now have an efficiently computable mixture of environments (ξ_{FAC}) to learn the true environment μ , and a method for searching in a given environment (ρ UCT). Combined, by instantiating ρ UCT with $\rho = \xi_{FAC}$, we get **MC-AIXI-CTW**, an approximation to the AIXI agent.

MC-AIXI-CTW action (moving horizon m)

$$a_t^* = \operatorname{argmax}_{a_t} \sum_{x_t} \cdots \max_{a_{t+m}} \sum_{x_{t+m}} \left[\sum_{i=t}^{t+m} r_i \right] \sum_{S \in \mathcal{C}_{D_1} \times \cdots \times \mathcal{C}_{D_k}} 2^{-\sum_{i=1}^k \Gamma_{D_i}(S_i)} P_S(e_{1:t+m} \| a_{1:t+m})$$

Compare with the AIXI policy (Theorem 7.4.2), rewritten for a moving horizon:

$$a_t^* = \operatorname{argmax}_{a_t} \sum_{x_t} \cdots \max_{a_{t+m}} \sum_{x_{t+m}} \left[\sum_{i=t}^{t+m} r_i \right] \sum_{\nu \in \mathcal{M}_{sol}} 2^{-K(\nu)} \nu(e_{1:t+m} \| a_{1:t+m})$$

The only differences: MC-AIXI-CTW uses a model class of a *product of prediction suffix trees* (Def. 4.3.11) instead of *all* semicomputable environments $\mathcal{M}_{sol}$ (Def. 3.7.1), a prior based on the tree *model cost* (Def. 4.3.20) instead of Kolmogorov complexity (Def. 2.7.3), and a *moving horizon* m to keep planning tractable.

Practical Notes on MC-AIXI-CTW

- Using ξ_{FAC} in place of the true environment μ for MCTS sampling was shown in [VNHS10] to converge to optimal behavior.
- Let m now be the maximum lifetime of the agent, as in the original AIXI model (the maximum number of interactions with the environment). Imagining a *changing* horizon h_t during interaction, let $h_{\max} = \sup_t h_t$ be the maximum of such horizons. We assume $m \gg h_{\max}$.
- One shortcoming of MC-AIXI-CTW is that the agent does not consider environments using information beyond the length- k context – this is (partially) addressed by later extensions such as Φ -AIXI-CTW (Chapter 14).
- In practice, when the agent has had limited interaction with the environment, CTW probability estimates are poor – there is little information to build estimates from. It can help to start with an ϵ -greedy exploration strategy to boost exploration, then switch back to UCT afterwards. This initial forced exploration differs from UCT's own exploration, which is only *guaranteed* to explore enough in the limit, but can be quite slow in practice to reach reasonable behavior.

Theorem 12.4.1: Upper Bound on FAC-CTW Value Error

Theorem 12.4.1 (Upper bound on FAC-CTW value error [VNH⁺11])

Using FAC-CTW (Algorithm 12.11), for every policy π , if the true environment μ is expressible as a product of k prediction-suffix trees $(\mathcal{S}_1, \Theta_1), \dots, (\mathcal{S}_k, \Theta_k)$, for all $b \in \mathbb{N}$, we have

$$\sum_{t=1}^m \mathbb{E}_{\mu}^{\pi} \left[\left(V_{\xi_{\text{FAC}}}^{\pi, m_t}(h_{<t}) - V_{\mu}^{\pi, m_t}(h_{<t}) \right)^2 \right] \leq 2h_{\max}^3 \left[\sum_{i=1}^k \Gamma_{D_i}(\mathcal{S}_i) + \left(\frac{1}{2} \log_2 m + 1 \right) \sum_{j=1}^k |\mathcal{S}_j| \right]$$

where $V_{\mu}^{\pi, m_t}(h_{<t})$ is the moving-horizon value of the policy (Definition 6.6.1), and $h_{\max} := \sup_t h_t$ is the maximum of the horizons $h_t := m_t - t + 1$.

Plain-English summary. The left side sums, over the agent's whole lifetime m , the expected *squared error* between the FAC-CTW value estimate and the true value of following policy π . The right side bounds this error in terms of: the complexity $\Gamma_{D_i}(\mathcal{S}_i)$ of each true component tree (simpler truth $\Rightarrow$ smaller bound), the number of leaves $|\mathcal{S}_j|$ of each tree, and $h_{\max}^3$, the maximum planning horizon cubed. It says the total squared error accumulated is *bounded*, not unbounded – so the algorithm's performance is likely acceptable for many practical applications, and gives a theoretical handle on how tree complexity trades off against error.

Consequences of Theorem 12.4.1

- The bound depends on the maximum planning horizon $h_{\max}$, the structure (complexity and size) of the prediction-suffix trees that express the true environment, and the agent's lifetime m .
- **Corollary:** the *average* expected squared difference between $V_{\xi_{FAC}}^{\pi, m_t}$ and V_{μ}^{π, m_t} tends to zero at rate $O(\frac{\log m}{m})$. This implies that for a sufficiently long lifetime m , the value-function estimates using ξ_{FAC} converge to those of μ , for *any* policy π – crucially, this includes the very policy that MC-AIXI-CTW is approximating: the fixed-horizon expectimax with respect to ξ_{FAC} .
- The FAC-CTW algorithm leverages prediction-suffix trees to build a *compact* representation of the environment, enabling efficient planning and learning, while this theorem gives a rigorous performance guarantee that practitioners can rely on when choosing whether MC-AIXI-CTW is appropriate for a given reinforcement-learning task.

Python: A Mini MC-AIXI-CTW Loop

```
# Toy end-to-end MC-AIXI-CTW-style loop on Example 12.3.2: reward=1 iff
# a_t == o_{t-1}. Learn  $P(r_t=1 \mid \text{context})$  with a KT estimator (the
# "learning" component); pick a_t via a 1-step rollout/expectimax
# stand-in for rho-UCT (the "searching" component).
import numpy as np
rng = np.random.default_rng(5)

kt = {True: [1, 1], False: [1, 1]} # [#reward=0, #reward=1] per context

def predicted_reward(match):
    a, b = kt[match]
    return b / (a + b) #  $P(\text{reward} = 1 \mid \text{context})$ 

o_prev = rng.integers(2)
total_reward, correct_actions = 0, 0
for t in range(500):
    # Search: try both actions, pick the one with higher *predicted*
    # reward under the learned model (a 1-step expectimax stand-in).
    q0 = predicted_reward(match=(0 == o_prev))
    q1 = predicted_reward(match=(1 == o_prev))
    a_t = 0 if q0 >= q1 else 1

    o_t = rng.integers(2)
    r_t = 1 if a_t == o_prev else 0
    total_reward += r_t
    correct_actions += int(a_t == o_prev)

    kt[a_t == o_prev][r_t] += 1 # Learning: update KT estimator used
    o_prev = o_t

print(f"average reward per step: {total_reward/500:.3f}")
print(f"fraction of optimal actions: {correct_actions/500:.3f}")
# -> both converge to 1.0: the agent learns "match o_{t-1}" is optimal
```

12.5 Experiments: Overview

MC-AIXI-CTW was implemented and tested by [VNHS10] on a wide variety of environments, performing close to optimal on almost every one. All environments are MDPs, but the underlying *state* of the MDP is often not directly visible to the agent – it receives only an *observation* o generated from the state via some observation function $\phi : \mathcal{S} \rightarrow \mathcal{O}$.

- An environment has **perceptual aliasing** if ϕ is non-injective, i.e. multiple states can map to the same observation, so the agent cannot uniquely identify the state from the observation alone (e.g. two different maze positions that look identical to the agent).
- An environment's observations are **uninformative** if the optimal policy does not depend on the observations at all – the agent must learn purely from rewards (e.g. matching pennies, Section 10.2.3, where the agent receives no useful observation, only a payoff).

The next few slides describe each tested environment in turn; Section 12.5.2 then shows the resulting learning curves (Figure 12.9).

Figure 12.4: Environment Properties

Environment	$ \mathcal{A} $	$ \mathcal{O} $	Aliasing	Stochastic $\mathcal{O}$	Uninformative $\mathcal{O}$
1D-Maze	2	1	yes	no	yes
Cheese Maze	4	16	yes	no	no
Tiger	3	3	yes	yes	no
Extended Tiger	4	3	yes	yes	no
4×4 Grid	4	1	yes	no	yes
TicTacToe	9	19683	no	no	no
Biased RPS	3	3	no	yes	no
Kuhn Poker	2	6	yes	yes	no
POCMAN	4	2^{16}	yes	no	no

Despite many of these environments being simplistic (with an often-obvious optimal policy), recall the agent has *no* prior knowledge of the environment beyond the size of the action and percept spaces, and no time to learn ahead of time – it must learn online while simultaneously trying to act well.

A Digression: Illegal Actions

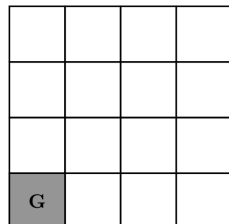
Under the current framework, the agent can take *any* action at any time, including “illegal” ones. An alternative would provide the agent with a legal-action function $\phi : \mathcal{H} \rightarrow 2^{\mathcal{A}}$ that restricts choices, but this ruins the universality of MC-AIXI-CTW (it requires hand-coding the rules of the game into ϕ , extra side information the agent otherwise has to discover for itself).

- If illegal moves are only penalized at the end of a fixed-length game, it can take the agent a very long time to learn the difference between illegal and merely bad moves. If an illegal action instead triggers an immediate penalty *and* a game restart, even a short-sighted agent quickly learns to avoid illegal moves.
- How large should the penalty be? An illegal action must be at least as bad as a loss (otherwise the agent could “cheat” its way out of a doomed position), but perhaps not *worse*. A convenient fix: whenever the agent plays an illegal action, return a percept (o, r) where o is the same observation as the last cycle and r is the lowest possible reward $r \in \mathcal{R}$; the environment never actually receives the illegal action, so the agent stays in the same position (apart from the minimal reward received). This makes illegal actions harmless to explore and learn from, but never advantageous.
- This trick requires knowing the true environment μ (often true when we design the test environment, even though μ is unknown to the agent itself).

12.5.1 Environments: 1D Maze and 4×4 Grid

1D Maze. A 1×4 POMDP grid-world with one goal state, actions $\mathcal{A} = \{left, right\}$, and a single (uninformative) observation $\mathcal{O} = \{none\}$. Each action moves the agent one cell; attempting to leave the grid has no effect. Walking into the goal teleports the agent to a random non-goal cell and gives reward 1; all other times reward is 0.

4×4 Grid. An extension of the 1D maze to two dimensions: the agent starts in a random cell, the single goal is the bottom-left cell (Figure 12.5), actions are the four cardinal directions, reward is 0 for moving to a non-goal cell (or attempting to leave the grid, which has no effect), and 1 for reaching the goal cell (after which the agent is teleported to a random non-goal cell). As with the 1D maze, observations are trivial ($\mathcal{O} = \{none\}$): the agent



must learn the navigation policy purely from the reward signal.

Environments: Cheese Maze

The **Cheese Maze** is a more complicated two-dimensional grid-world with one goal state. The agent has 4 actions (*up*, *down*, *left*, *right*) and 16 observations $\mathcal{O} = \mathbb{B}^4$: each bit of the observation indicates whether a wall is directly adjacent to the north, east, south, or west of the agent, respectively.

There are three rewards: -10 for bumping into a wall, $+10$ for finding the goal state (“cheese”), and -1 for moving into a state that is not a wall or the goal (to incentivize finding the cheese quickly).

Whenever the agent enters the cheese cell, it is teleported to a random non-cheese cell.

Since the environment has *partial aliasing*: if the agent starts in a cell and observes 0101, there is not enough information to determine whether the cheese is directly below, or the agent is in one of the two side corridors – it needs to move up and then down to disambiguate and find the cheese.

	1001	1010	1000	1010	1100	
	0101		0101		0101	
	0111		⁰¹¹¹ G		0111	

Environments: Tiger and Extended Tiger

Tiger. The agent faces two doors: behind one is gold, the other a tiger. Three actions: *listen* (tells the agent which door the tiger is behind, correct with probability 0.85), *open door 1*, *open door 2*. Opening a door gives reward -100 if the tiger is found, 10 if the gold is found. *Listen* gives reward -1 (to discourage listening forever, though the agent is not forbidden from doing so). Once a door is opened, the doors close and are reshuffled.

Extended Tiger. Adds: the agent is either sitting or standing, starting seated. Two additional actions, *stand up* and *sit down*. The agent can only listen while sitting, and only open doors while standing. Any illegal action (opening a door while sitting, standing while already standing, etc.) gives reward -10 . The reward for finding the gold is now 30 . The environment resets once any door is opened.

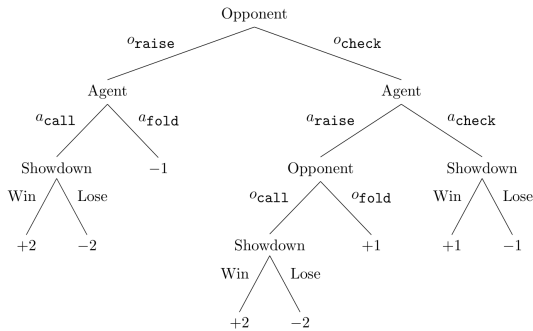
Environments: TicTacToe and Biased RPS

Tic Tac Toe. The well-known game against an opponent who chooses actions uniformly at random. The agent has 9 actions: placing a piece in one of the nine squares. The observation space is the current board state. Reward 2 for winning, 1 for a draw, -2 for losing, -3 for an illegal move, 0 otherwise. Illegal moves are punished *more* than losing to encourage the agent to learn legal moves and prefer losing to playing illegally.

Biased RPS (Rock-Paper-Scissors). The opponent is *biased*: if they won the last round playing rock, they choose rock again; otherwise they choose uniformly at random. The agent needs to learn and exploit this bias. Reward 1 for winning, 0 for a draw, -1 for losing.

Environments: Kuhn Poker

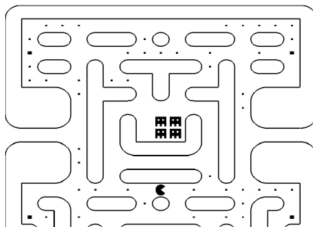
Kuhn poker [Kuh50] is a simplified poker with a 3-card deck: King > Queen > Jack. The opponent plays first, the agent second. Observations are the card held plus the opponent's actions. Both players *ante* one chip. One betting round: opponent *raises* (bets another chip) or *checks* (passes). If raised, the agent *calls* (matches) or *folds* (surrenders). If checked, the agent may raise or check (to which the opponent must then call or fold). If a player folds, the other wins the pot minus the chips they staked; otherwise a *showdown* occurs and the higher card wins the pot. This version is simple enough that a mixed **Nash equilibrium** strategy is known for both players – the opponent plays this Nash strategy, and the optimal strategy for the agent is also the Nash strategy, giving an average return of 1/18 reward per hand.



Environments: POCMAN

The most complex environment tested is **POCMAN** [SV10], a partially observable version of the classic arcade game Pac-Man. The agent navigates a 17×17 maze (Figure 12.8) and eats food pellets. Four ghosts roam the maze: initially at random, but once within Manhattan distance 5 of Pac-Man they aggressively pursue for a short duration. Pac-Man only receives a **4-bit** observation of the wall configuration at its current location (as in the Cheese Maze), a 4-bit observation of whether a ghost is visible (via direct line of sight) in each of the four cardinal directions, a 3-bit observation indicating whether a food pellet exists within Manhattan distance 2, 3, or 4, a 4-bit observation of food visible via direct line of sight, and a single bit indicating whether Pac-Man is under the effect of a power pill (which allows eating ghosts for the next 100 steps).

Rewards: -1 per movement action, -10 for running into a wall, $+10$ per food pellet eaten, -50 if killed by a ghost (episode resets), $+30$ for eating a ghost under a power pill, $+100$ for collecting all the food (cumulative if multiple events occur simultaneously). The episode resets if the agent is caught or collects all food.



12.5.2 Empirical Performance

In each environment except POCMAN, [VNHS10] compared MC-AIXI-CTW against two baselines: **U-Tree** [UV98] (which learns a compact state representation from the raw history stream) and **Active-LZ** [FMRW10a] (which uses a Lempel-Ziv-based prediction scheme combined with dynamic programming for control). MC-AIXI-CTW outperformed both baselines in every environment tested. Figure 12.9 (shown earlier) summarizes performance across environments, with reward normalized so that an average reward of 1 per cycle corresponds to the optimal policy. On every environment except POCMAN, MC-AIXI-CTW eventually learns to play optimally – POCMAN's much larger state and observation space (2^{16} possible observations) makes learning slower and full convergence harder to reach within a practical number of interactions.

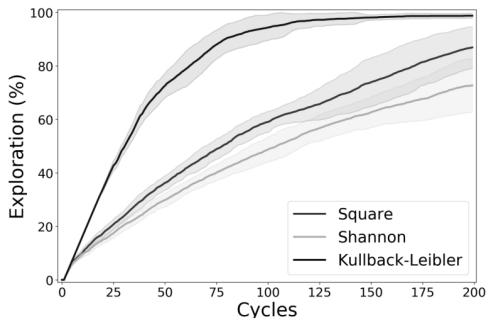
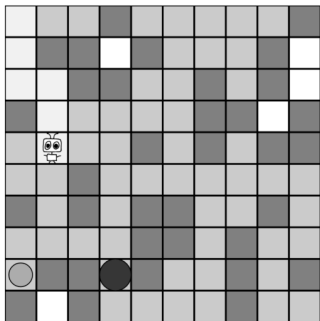
12.6 AIXIjs: A JavaScript Testbed for AIXI Approximations

AIXIjs [ALH17, Asl17] is a toy implementation, in JavaScript, of various approximations of AIXI from Chapters 7, 9, and 12 of this book. It is an easy-to-use testbed for experimentally comparing (approximations of) AIXI and other universal agents, such as **Knowledge-Seeking Agents**, **BayesExp**, and **Thompson sampling**. Several environments are implemented, including various gridworlds and MDPs [Put14].

AIXIjs has been used to study wireheading in partially embedded agents [MSZ19], the effects of generalized discount functions [LALH17], and how new universal-AI agents perform experimentally compared to other UAI agents [CCH19, CHC21].

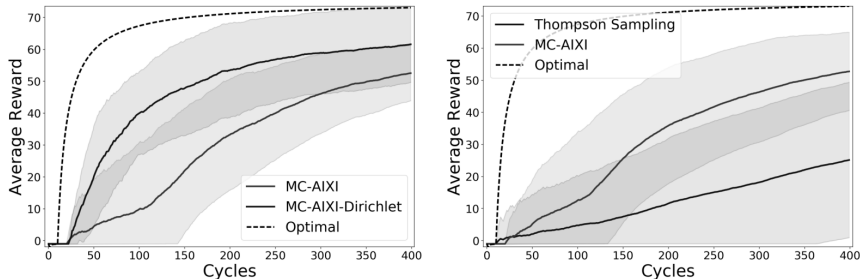
This chapter's comparison uses two choices of model class over a 10×10 gridworld containing a single reward dispenser (which gives reward with probability 0.75): (1) a class of possible reward-dispenser *locations* – used by the base AIXI approximation MC-AIXI-CTW; and (2) a Dirichlet distribution over grid cells the agent can visit – used by the knowledge-seeking agents (Section 9.3). For the UCT search, 600 samples are used with a planning horizon of 6; each agent's performance is averaged over 50 simulations.

Figure 12.10: Knowledge-Seeking Exploration in AIXIjs



(Left) a gridworld in AIXIjs: darker gray indicates a higher belief of potential reward at that cell. (Right) exploration percentage over time for three knowledge-seeking agents: *Square*, *Shannon*, and *Kullback-Leibler* (KL). Performance in terms of intrinsic motivation is highly model-dependent: for the Dirichlet base model class, the information-seeking (KL) policy outperforms entropy-seeking (Shannon) in this stochastic environment; the curves for the location-based model class look similar to Shannon but only reach $\approx 75\%$ exploration.

Figure 12.11: MC-AIXI vs. Thompson Sampling



(Left) average reward of MC-AIXI using the *location* model class (thin line) vs. the *Dirichlet* model class (thick line), both vs. the theoretical optimum. (Right) MC-AIXI typically outperforms Thompson sampling: both share MCTS horizon $m = 6$ and budget 600 samples/action, yet Thompson sampling is unreliable, with low mean reward and high variance (with horizon 10 and budget 100, the gap narrows but remains significant). Interestingly, Thompson sampling is *proven* asymptotically optimal while MC-AIXI-CTW is not – likely its extra exploration hurts it in this finite-sample regime relative to MC-AIXI-CTW.

Python: Knowledge-Seeking Exploration Bonuses

```
# Illustrative (not from the book): compare three intrinsic exploration
# bonuses used by knowledge-seeking agents in AIXIjs, for a simple belief
# update over "is there a reward dispenser in this cell?" (Bernoulli(theta)).
import numpy as np

def square_bonus(p_before, p_after):
    return (p_after - p_before) ** 2

def shannon_bonus(p_before, p_after):          # entropy-seeking (Shannon)
    def H(p):
        p = np.clip(p, 1e-9, 1 - 1e-9)
        return -(p * np.log(p) + (1 - p) * np.log(1 - p))
    return H(p_after) - H(p_before)

def kl_bonus(p_before, p_after):               # information-seeking (KL)
    p_before = np.clip(p_before, 1e-9, 1 - 1e-9)
    p_after = np.clip(p_after, 1e-9, 1 - 1e-9)
    return p_after * np.log(p_after / p_before) + \
        (1 - p_after) * np.log((1 - p_after) / (1 - p_before))

p_before = 0.5          # uniform prior belief
p_after = 0.9           # after observing strong evidence of a dispenser
for name, fn in [("Square", square_bonus), ("Shannon", shannon_bonus), ("KL", kl_bonus)]:
    print(f"{name:8s} bonus = {fn(p_before, p_after):+.4f}")

# Square    bonus = +0.1600
# Shannon   bonus = -0.2081    (surprisingly negative: entropy *decreased*)
# KL        bonus = +0.5108    (KL divergence is always non-negative)
```

12.7 Discussion

- The Monte Carlo AIXI Context-Tree-Weighting (MC-AIXI-CTW) agent represents the initial attempt to approximate AIXI in a way that scales to more complex environments than the exhaustive-search AIXI-MDP agent of Chapter 11.
- By employing the CTW method, ξ_{FAC} constructs an *efficient* Bayesian mixture, yielding a manageable version of AIXI's mixture ξ – turning a double-exponential intractability into an $O(D)$ -per-bit update.
- As discussed in Chapter 5, various extensions of CTW have been proposed, each enhancing different aspects of the original method (handling different context structures, different priors, etc.). These improvements can be readily integrated into the MC-AIXI-CTW framework, replacing the original CTW component and potentially improving the agent's overall performance.
- Together, MCTS (searching) and CTW (learning) show that both halves of the AIXI equation – planning given a model, and learning a model from data – admit efficient, practical approximations, and that combining them yields an agent that performs well across a strikingly diverse range of environments, entirely from scratch.

12.8 Exercises I

1. [C07] Assume a bandit problem has a unique optimal action a^* . Prove that an ε -greedy agent converges to a policy that chooses a^* with probability $1 - \varepsilon(1 - \frac{1}{|\mathcal{A}|})$.
2. [C25] In UCT, derive the worst-case-regret bound of using $\ln^2 N(h)$ or $\sqrt[3]{N(h)}$ (or other functions in $o(\sqrt{N(h)})$). How do they compare to the bound of $\ln N(h)$?
3. [C30] Prove Theorem 12.2.6.
4. [C15] Run an experiment comparing the performance of BayesExp and Thompson sampling agents in gridworlds on the AIXIjs website.
5. [C12] Prove that we can always apply a positive affine transformation to the rewards to ensure they are always non-negative, without changing the optimal policy. See also Exercise 15.5.
6. [C25] Implement ρ -UCT together with FAC-CTW to replicate MC-AIXI-CTW, and graph the performance against each environment (Figure 12.9). A reference implementation can be found at <https://jveness.info/software/default.html>.
7. [C25] Investigate why leaf parallelization performs poorly compared to other methods for parallelizing MCTS (Section 12.2.6).

12.8 Exercises II

8. [C28] Investigate the extensions of CTW (Chapter 5) and use them in place of CTW to define ξ_{FAC} . Does this improve the MC-AIXI-CTW model?
9. [C20] Train MC-AIXI-CTW on a simple environment (e.g. 1D Maze or Tiger) and then continue on another simple environment. Does MC-AIXI-CTW perform better or worse on the second domain? Repeat with different pairs of environments.
10. [C18] For the UCT action selection, test alternative choices of the constant C . Show that if C is sufficiently large UCT will fail.
11. [C20] Use the CTW algorithm to learn a fast approximation of the policy and use it for the rollout in ρ UCT. What is gained/lost by using this in place of a random policy?
12. [C32] Prove Theorem 12.4.1.
13. [C25] Generalize FAC-CTW to $\mathcal{P}$ -context trees defined in Section 9 of [VNH⁺11].

12.9 History and References (1/5)

This chapter is primarily based on [VNH⁺11], where CTW (for learning) and the UCT algorithm (for planning) are first combined. In [BVB13], an agent was constructed for Atari 2600 games by recursively factoring the large observation space into smaller, manageable sub-problems, with a Bayesian average taken over the result – working similarly to CTW. [VSH12] surveys possible ways to extend CTW, particularly for the active (reinforcement learning) case; FAC-CTW, introduced in [VNHS10], is one novel example. The AIXI_ls testbed was developed in [Asl17, ALH17], with various approximations of AIXI discussed in this book and various toy environments implemented.

AIXI approximations. Several other attempts have been made to study the computational properties of AIXI:

- AIXI_l_t [Hut05b, Chp. 7] (Section 13.2) uses the best policy, as measured by a provable lower bound on the value function, bounded in space l and time t . It is based on the provably fastest algorithm for all well-defined problems [Hut01b, Hut02b], related to **Levin Search** [Lev73c], which solves any inversion problem optimally within multiplicative overhead but is impractical in general (though can be applied successfully in some cases [Sch97b, SZW97, Sch03, Sch04]).

12.9 History and References (2/5) I

- **Self-AIXI** [CGMH⁺23] (Section 9.5) is a modification of AIXI that avoids planning by self-predicting its own actions; its practical instantiation Self-AIXI avoids the expensive MCTS used in MC-AIXI-CTW.
- In domains with very small action/percept spaces and restrictive assumptions on the model class, universal learning is computationally feasible via brute-force search [PH06b] (Chapter 11). The behavior of AIXI-MDP is compared with a universal predicting-with-expert-advice algorithm [PH05b] in repeated 2×2 matrix games, and shown to exhibit different behavior.
- In [Pan08], a Monte Carlo algorithm samples from a time-bounded version of the universal Solomonoff prior. A closely related algorithm samples from a **speed prior** [Sch02b, FLH16], a universal semimeasure like the Solomonoff prior that also considers the number of time steps needed to compute a result.
- **Φ -AIXI-CTW** [YZWN22] (Chapter 14) abstracts the complete history (as in Chapter 14) and then applies CTW to the abstracted history, enabling environment model classes with arbitrary dependence on the history.
- **DynamicHedgeAIXI** [YZNH24] dynamically adds (and subtracts) models via a time-adaptive prior built from a variant of the Hedge algorithm – the latest and richest direct approximation of AIXI at time of writing, with good theoretical and practical guarantees.

12.9 History and References (2/5) II

- An alternative approach to approximating Bayes-optimal agents like AIXI was developed in [MDM⁺20], where it was shown that *meta-training* can approximate Bayes-optimality.

12.9 History and References (3/5): UCB, MCTS, USM/U-Tree

Upper Confidence Bounds and Monte Carlo Tree Search. UCB was first introduced in [ACBF02]; it has since been extended to contextual bandits [LS20, Part V] with linear, non-linear, and gated linear function approximation [SHB⁺20]. Extending UCB from bandits to MDPs (i.e. UCT) was done in [KS06]. UCB has had many successful practical uses, including the game Go [GW06], and learning to play new games from a provided description of the rules [FB08]. [GS07] modified UCB to combine online-learned statistics with offline learning via $TD(\lambda)$. Improvements to the rollout stage of MCTS include a 1-ply rollout-based planning technique for noisy stochastic domains [BC99], knowledge-based rollout policies in Go [GWMT06], and learning the rollout policy for adversarial games [ST09]. MCTS was used to adjust a heuristic for chess game-tree search [VSBU09]. For large partially observable domains, the **POMCP** algorithm combines the agent's belief state with MCTS from the state the agent believes itself to be in [SV10].

USM and U-Tree. An early influential work is the **Utile Suffix Memory** (USM) algorithm [McC96], which partitions the agent's history space into distinct states, one per leaf of a suffix tree, each with an associated Q-value updated incrementally much like Q-learning [WD92]. The suffix tree grows incrementally, splitting a leaf when its history sequences show statistically different Q-values; USM does not deal effectively with noisy environments (extensions in [SB04, Sha07]).

12.9 History and References (4/5): U-Tree, Universal RL, BLHT

U-Tree [McC96] is an online agent algorithm that discovers a compact state representation from raw experience. Unlike USM, U-Tree can discriminate between *individual components* within an observation, letting it handle larger observation spaces and ignore irrelevant components. Each state is a leaf of a suffix tree mapping history sequences to states; the representation is refined via a heuristic built around the Kolmogorov-Smirnov test [BZ14], limiting the tree's growth to nodes that improve future-reward prediction. Value iteration updates the value function at each time step.

Universal RL. Active-LZ [FMRW10a] combines a Lempel-Ziv-based prediction scheme [ZL77] with dynamic programming for control, producing an agent provably asymptotically optimal in k -Markov environments. It builds a context tree (distinct from the CTW context tree) with accumulated transition statistics and a value-function estimate at each node, refined over time. MC-AIXI-CTW compares favorably to Active-LZ [VNH⁺11, Sec. 7].

Bayesian approach to Learning History Trees (BLHT) [SHL97, SH98] uses symbol-level PSTs for learning and dynamic programming for control. BLHT uses the **MAP** (maximum a posteriori) model for prediction, as opposed to CTW's full Bayesian mixture, which admits a much stronger convergence result; CTW's prior also weights models by complexity (Occam's razor), instead of the uniform prior over PST models used by BLHT.

12.9 History and References (5/5): PSRs, MBRL, CnC, Quantum AIXI I

Predictive State Representations (PSRs) [LS01, RGT04, SJR12] maintain predictions of future experience as a vector of probabilities for the outcomes of various *tests* (predicates about future observations given particular action sequences). If every test's outcome were known, the environment's dynamics would be known; a PSR is a sufficient subset of these predictions. Exact PSRs are impractical in large domains, so approximation is typically required (e.g. kernel-based PSRs [BSG11]; temporal-difference networks [ST04], where an *answer network* predicts future observations and is updated via temporal-difference learning). In model-based Bayesian RL [Str00b, PVHR06, RCdP07, PV08], a distribution over (PO)MDP parameters is maintained; in contrast, MC-AIXI-CTW maintains an *exact* Bayesian mixture of PSTs.

Compress and Control (CnC) [VBH⁺15, DRW⁺24] decomposes the distribution of the return z (rather than predicting observations/rewards one step at a time) via $\rho(z|s, a) \triangleq \rho(s|z, a)\rho(z|a)$; using CTW leads to consistent estimation [VBH⁺15]. This is a compression-based method for both policy evaluation and on-policy control.

Quantum AIXI. [CH20a] first tackled whether quantum computing can speed up approximations of Solomonoff induction and AIXI; speedups were found but not the exponential speedup quantum computing achieves elsewhere. [Sar21, SAAGB21] instead consider a model class of *quantum-computable* (semi)measures, defining a Knowledge-Seeking Agent (Section 9.3) variant over this model class.

12.9 History and References (5/5): PSRs, MBRL, CnC, Quantum AIXI II

Related approximations & AI search. Compress and Control, Representational MDL [PR12], Feature Reinforcement Learning (Chapter 14), and low-Kolmogorov-complexity policy search [SJ21] all embody the spirit of UAI without being direct AIXI approximations. MCTS [KS06] is one instance of the long tradition of AI search: (iterative-deepening) A^* [HNR68, Kor85], breadth-first search [Zus72], depth-first search [Tre76] (see [EH15a, EH15b] for a comparison), and Levin (tree) search [OHL23], among many variants.

Chapter Summary

- **Searching:** exact expectimax is intractable ($O(|\mathcal{A} \times \mathcal{E}|^m)$); **Monte Carlo Tree Search** approximates it via sampled rollouts, guided by **UCB**-style action selection (**UCT**) that provably converges and provably requires the $\ln N(h)$ exploration term to avoid getting permanently stuck.
- **Learning:** the exact AIXI mixture ξ over all semicomputable environments is intractable; **CTW** computes an analogous mixture over context trees in $O(D)$ time. Extending CTW to actions (**AC-CTW**) and factoring it per percept bit (**FAC-CTW**) preserves type information the agent needs to tell rewards from observations.
- **Putting it together:** instantiating ρ UCT with $\rho = \xi_{FAC}$ gives **MC-AIXI-CTW**, a practical, efficiently computable approximation of AIXI with a formal error bound (Theorem 12.4.1) and strong empirical performance across a wide range of POMDP-style environments – from simple mazes to Kuhn poker to POCMAN.
- MC-AIXI-CTW is not the final word: Φ -AIXI-CTW, DynamicHedgeAIXI, Self-AIXI, and meta-trained approximations all push further along the same basic recipe of *tractable learning* + *tractable search* $\approx$ AIXI.